

An Efficient Minimum Spanning Tree Algorithm

Abdullah-Al Mamun

Computer Science and Engineering
University of Connecticut

Connecticut, USA

Email: abdullah.am.cs@engr.uconn.edu

Sanguthevar Rajasekaran

Computer Science and Engineering
University of Connecticut

Connecticut, USA

Email: rajasek@engr.uconn.edu

Abstract—Finding minimum spanning trees (MST) in various types of networks is a well-studied problem in theory and practical applications. A number of efficient algorithms have been already developed for this problem. In this paper we present an efficient algorithm, namely *Edge Pruned Minimum Spanning Tree (EPMST)* algorithm, which combines ideas from randomized selection, Kruskal's algorithm and Prim's algorithm. The algorithm has a superior performance relative to the best-known algorithms especially when the graph is not very sparse. Specifically, EPMST outperforms a recently devised efficient algorithm on a wide range of input graphs.

I. INTRODUCTION

The minimum spanning tree (MST) problem of a connected graph $G = (V, E)$ is to find a spanning tree with minimum total edge weight. This problem has been extensively studied and is a subproblem of many well known network problems. It has practical applications in wireless networks and VLSI layout design [1]–[3], many graph problems such as ear decomposition, connectivity checking etc. [4]–[6], ovarian and bronchial cancer detection and many other medical diagnoses [7]–[10], network evolution [11], and so on.

A large number of algorithms have been devised to find minimum spanning trees in graphs. Boruvka's algorithm finds the cheapest edges from each component until only one component is left. The algorithm runs in $\mathcal{O}(|E| \log |V|)$ time where V is the set of vertices and E is the set of edges. Jarnik-Prim algorithm [12], [13] starts with a single node and grows the tree by adding one edge at a time until all vertices are in the tree. Implementation of the algorithm uses efficient priority queues, and the runtime is mostly dependent on the specific implementation of a priority queue. It takes $\mathcal{O}(|E| \log |V|)$ time if a binary heap is used to realize a priority queue. A Fibonacci heap version takes $\mathcal{O}(|E| + |V| \log |V|)$ time. Kruskal's algorithm [14] is a greedy algorithm which finds a minimum spanning forest. It sorts the edges in increasing order of their weights. It starts with forest of $|V|$ forests where each tree is a single node (corresponding to a vertex in the graph). From thereon it tries to add one edge at a time into the forest. If the edge does not cause a cycle it is added in which case the number of trees decreases by one. When the forest has only one tree, the algorithm terminates. The running time of the algorithm is $\mathcal{O}(|E| \log |V|)$. More efficient algorithms have been developed based on these algorithms.

Prim's algorithm performs better than Kruskal's algorithm for dense graphs whereas Kruskal's algorithm is faster for

sparse graphs. The algorithm we propose in this paper, Edge Pruned MST (EPMST), exploits the ideas of both of these algorithms. In particular, EPMST uses Kruskal's algorithm on a small subset of the edges. Then if needed, it uses Prim's algorithm on a denser compact graph. EPMST employs random sampling to choose the subset of edges.

The rest of this paper is organized as follows. In Section 2 we provide a brief summary of existing MST algorithms. In section 3 we describe a recently devised efficient algorithm, Filter-Kruskal [15], briefly. In this section we also present and analyze our algorithm, EPMST. Section 4 presents an experimental comparison between EPMST and Filter-Kruskal over a wide variety of input graphs.

II. RELATED STUDIES

Algorithms developed by Boruvka, Kruskal, and Prim provide the base for a large number of efficient algorithms. Since Kruskal's algorithm works on sorted edges and MST edges are likely to be among the $\mathcal{O}(n \log n)$ smallest weight edges, partial sorting of edges work greatly to find out lighter edges [16], [17]. Filtering of edges connecting nodes in the same component makes the algorithm much faster [18]. Bucket sorting on edges has been also employed in Kruskal's algorithm which performs greatly for uniformly distributed random edge weights [19]. A recently devised Filter-Kruskal algorithm has an improved performance over a wide range of input types [15]. The Filter-Kruskal algorithm has been compared with some well-known fast algorithms and has been shown to yield a better performance. As a result, in this paper, we have chosen the Filter-Kruskal algorithm to compare with our algorithm.

III. METHODS

In this section we devise and analyze an efficient minimum spanning tree algorithm. The algorithm performs much better for not very sparse graphs. We have chosen the Filter-Kruskal algorithm to compare with as this is the latest well-known algorithm which has a better performance over previous algorithms.

A. Filter-Kruskal Minimum Spanning Tree Algorithm

Filter-Kruskal is a modified and extended version of Kruskal's algorithm which avoids sorting all of the input edges [15]. The algorithm runs Kruskal's algorithm on subsets of candidate edges repeatedly. To find these subsets, it chooses

a pivot edge which separates unused edges into two sets. The method is called on these sets recursively. Lighter sets are more probable edge sets. If all the required edges are not found in these sets, then it explores the heavier edge sets. Another important feature of this algorithm is that it chooses only those edges whose end points are in two different components.

The algorithm selects $c|V|$ number of edges each time it invokes Kruskal's algorithm, where c is a constant. It chooses the median of a random sample of size $\sqrt{k}$ where k is the size of an input segment as the pivot value. The running time of the algorithm is $O(m + n \log n \log \frac{m}{n})$ for not too sparse graphs, where $m = |E|$ and $n = |V|$.

[15] compares Filter-Kruskal algorithm with Kruskal, qKruskal - a modified version of Kruskal, qJP - Jarnik-Prim implementation combined with Paredes's quick-Heap priority queue, and pJP - Jarnik-Prim implementation with pairing heaps. This algorithm has been tested over a wide range of edge densities. The comparison has been done using five types of graphs, namely random graphs with random edge weights, difficult instances where decreaseKey method of JP algorithm is called for almost every edge, random geometric graphs, lollipop graphs and image segmentation instances as real instances.

Filter-Kruskal performs better than all other algorithms for random graphs with random edge weights. When the edge density is high, it is outperformed only by pJP. For difficult instances, JP and Filter-Kruskal perform in the same way, but Kruskal and qKruskal do better. Filter-Kruskal outperforms all the others for random geometric graphs with 2^{16} nodes. pJP and qJP run faster than Filter-Kruskal by a small margin for lollipop graphs. For real instances, this algorithm performs the best.

B. Edge Pruned Minimum Spanning Tree (EPMST) Algorithm

Our EPMST algorithm uses ideas of randomized selection, Kruskal's MST and Prim's MST algorithms. The algorithm selects a portion of the input edges. For dense graphs with n vertices and m edges with random edge values, the $n \log n$ lightest edges are more probable candidates to be minimum spanning tree edges. Therefore the algorithm chooses the $n \log n^{\text{th}}$ lightest edge as the pivot using a randomized selection algorithm. Then it identifies the edges having weights no greater than that of the pivot edge. It uses Kruskal's algorithm on the graph with these edges only. Kruskal's algorithm finds a minimum spanning forest. If the number of trees in this forest is 1, then the algorithm has already found a MST for the input graph and hence terminates. Otherwise it labels the connected components in this forest as super vertices and finds spanning tree using Prim's algorithm.

C. Analysis of Algorithm EPMST

Lemma 1: The expected run time of EPMST on any weighted graph $G(V, E)$ is $O(|E| + |V| \log^2 |V|)$. Here the expectation is over the space of all possible inputs.

Algorithm 1 EPMST (Edge Pruned Minimum Spanning Tree)

Input: A set of sequencing reads and K (the desired substring length)

Output: A list of unique K -mers in the input with their counts

```

1: procedure EPMST
2:   Let  $G(V, E)$  be the input graph. Choose a pivot edge
   using a randomized selection algorithm. This pivot edge
   is the  $cn \log n^{\text{th}}$  lightest edge, where  $c$  is a constant
3:   Identify edges having weights no greater than that of
   the pivot edge. Let  $E'$  be the set of these edges
4:   Run Kruskal's algorithm on  $G'(V, E')$  to get a forest
    $F$ 
5:   Find the connected components in  $F$ 
6:   if the number of connected components in  $F$  is exactly
   one then
7:     the connected component is the desired spanning
     tree
8:   else
9:     construct a graph  $G''(V'', E'')$  where each con-
     nected component of  $F$  is a vertex in  $V''$ .  $(a, b) \in E''$  if
      $(a, b)$  is an edge in  $G$  that connects the two components  $a$ 
     and  $b$  and has the least weight from among all the edges
     connecting  $a$  and  $b$ 
10:    Run Prim's or Kruskal's algorithm on  $G''$ 
11:  end if
12:  Output edges used in Prim's or Kruskal's algorithm on
    $G(V, E)$ 
13: end procedure

```

Proof: Consider the random graph model $\mathcal{G}(n, m)$, where $n = |V|$ and $m = |E|$. In this model we assume that each set (out of the $\binom{n}{m}$ possible such sets) of m edges is equally likely [20]. Assume that the input is a random graph from $\mathcal{G}(n, m)$ in which the edge weights are chosen randomly.

Let q be the number of edges whose weights are less than that of the pivot in EPMST. We can think of the subgraph induced by these edges as a $\mathcal{G}(n, q)$. The reason is as follows. The edge with the least weight can be any one of the m edges with equal probability, the edge with the next least weight could be any one of the remaining edges with equal probability, etc. As a result, these q edges can be any q -subset of the m edges with equal probability. This implies that the subgraph induced by the q edges is indeed a $\mathcal{G}(n, q)$.

Theorem VII.3 in [20] implies that $\mathcal{G}(n, q)$ is connected with high probability when $q = \Omega(n \log n)$. This means that in EPMST when we call Kruskal's algorithm, there will be only one connected component with high probability.

EPMST finds the edge with the $n \log n^{\text{th}}$ smallest weight as the pivot, using a randomized selection algorithm. Finding the pivot among the m edges takes $O(m)$ time. Partitioning of the edges takes $O(m)$ time. Kruskal's algorithm then runs on a graph with n vertices and $O(n \log n)$ edges. Sorting within Kruskal's algorithm takes $O(n \log n \log n)$ time. Over-

all, Kruskal's algorithm takes this amount of time. With high probability, Kruskal's algorithm finds a minimum spanning tree. Therefore there is no need to create super vertices and run Prim's algorithm. Since this happens with a low probability, it follows that the expected run time of EPMST is $O(m + n \log^2 n)$. $\square$

If in EPMST, Kruskal's algorithm does not find the MST, an additional $O(m)$ time is needed to create super vertices. Kruskal's algorithm outputs a number of connected components. Experiments show that the number of generated components in most cases is very low. Therefore, Prim's algorithm on $G''(V'', E'')$ takes a negligible amount of time in practice.

D. Parallel EPMST algorithm

The more time consuming part of the sequential EPMST is partitioning of edges to choose a subset of edges for Kruskal's method as it requires $O(|E|)$ time to scan through all the edges. If the input graph is dense, this is the most time taking phase compared to the other portions of the algorithm. In the parallel version a master processor distributes the edges among all the other processors. Each processor chooses a set of random edges and sends them to the master. The master collects all the edges from the slaves and chooses a pivot edge from this random sample. The master then broadcasts this pivot to all the slaves. Then each slave identifies edges whose weights are less than that of the pivot and sends these edges to the master. The master runs the Kruskal's algorithm on the edges received from the slaves. Since the Kruskal's method takes a negligible amount of time compared with the partitioning (done by the slaves), currently we keep this method sequential by calling this only from the master. Other portions of the algorithm are trivial as they do not consume comparable time. For random dense graphs with 10,000 nodes the speed up we get is 2.13 using 4 cores in a single node. The speed up is 2.40 and 2.79 for graphs with 20,000 and 30,000 nodes, respectively. Speedup using more cores is not promising since Kruskal's algorithm run by the master takes time comparable to partitioning.

IV. EXPERIMENTS

We have implemented our EPMST algorithm in C++. We do not have access to any running version of Filter-Kruskal implementation. As a result, we have implemented this algorithm ourselves. In presenting timing results, we not only show the times taken by our implementation of Filter-Kruskal and EPMST but also the times reported in the Filter-Kruskal paper [15]. The timing results shown in [15] have been obtained on a machine with two AMD Opteron 2.0 GHz quad-core CPUs. We have run our implementations on a computer with 2.4 GHz Intel Core 2 Duo processors. We have thus scaled the running times (linearly) while reporting the timing results of [15]. It is true that this scaling may not be 100% accurate since factors such as cache memory size, main memory size, etc. also play a role. However with the absence of a running program for Filter-Kruskal, we feel that

this the best we could do. We also show timing results for our implementation of the Filter-Kruskal algorithm. GNU compiler and optimization level O3 have been used to compile our implementations. All the programs can be downloaded from: <https://github.com/abdullah009/MinimumSpanningTree>.

We have used synthetic graphs to test the algorithms as real graphs are hard to find. We have generated five types of graphs. [21] describes generation processes of some types of graphs. Random graphs are generated by varying edge values within a range randomly. We use difficult instances of graphs where decreaseKey method of Prim's algorithm is called for almost every edge. In geometric graphs, each point is connected with k nearest neighbors in the Euclidean space. A path of length $\frac{n}{2}$ connected with a random graph of $\frac{n}{2}$ nodes generates a difficult graph instance namely a lollipop graph. Our algorithm shows much better performance for dense graphs. We have used dense graphs having randomly generated edge values.

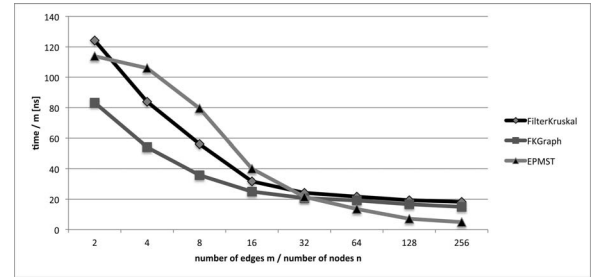


Fig. 1. Time per edge for random graphs with 2^{10} nodes and random edge weights.

We have generated sparse graphs with random edge weights. Two graphs with different numbers of nodes have been employed in our experiments. To compare the performance with the Filter-Kruskal algorithm, we plot two lines for these two algorithms in all the figures. Our algorithm is denoted as EPMST and our implementation of the Filter-Kruskal algorithm is denoted as FilterKruskal. Another line is drawn to show (a scaled version of) the times reported in the Filter-Kruskal paper [15]. This line is denoted as FKGraph. Fig. 1 displays the running times of the algorithms for 2^{10} nodes and different numbers of edges per node. We choose $2n$ as the kruskalThreshold value. We use these many edges for the Kruskal method in the Filter-Kruskal algorithm. We use $2n$ as the kruskalThreshold value for all the graphs. Fig. 1 shows that the Filter-Kruskal algorithm works better when the edge per node value is below 32. Beyond that point, EPMST outperforms the former one. The figure clearly shows that our algorithm is at least two times faster than Filter-Kruskal when the edge per node value is 256. The trend also indicates that the margin will be higher for denser random graphs.

Table I explains different phases of EPMST for random graphs with 2^{10} nodes and random edge weights. The table has eight columns, namely, number of edges per node, elapsed

TABLE I
ANALYSIS OF DIFFERENT PHASES OF EPMST FOR RANDOM GRAPHS WITH 2^{10} NODES AND RANDOM EDGE WEIGHTS

m/n	T_Select	Sample Size	T_Kruskal	Num Trees	T_SV	T_Kruskal2	T_Total
16	238	2079	307	43	104	6	655
32	279	2033	301	34	110	5	695
64	377	1807	269	52	181	10	837
128	432	1986	282	25	210	5	929
256	592	2463	341	13	355	4	1292

time for selection and partition methods, number of vertices used for Kruskal's algorithm, time spent on Kruskal's algorithm, number of trees in the forest after the Kruskal phase, time used for the creation of supervertices, time used on the second call to Kruskal's method and the total time spent. The table is self explanatory.

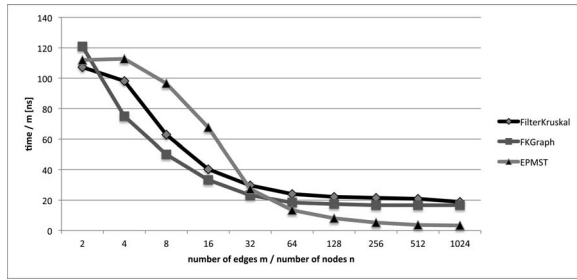


Fig. 2. Time per edge for random graphs with 2^{16} nodes and random edge weights.

Fig. 2 compares FilterKruskal and EPMST on random graphs with 2^{16} nodes. We see that these results are similar to the ones in Fig. 1. Again EPMST outperforms Filter-Kruskal when the number of edges per node is 16 or more. These figures also show that our implementation of Filter-Kruskal and the scaled values from the original paper of Filter-Kruskal algorithm follow almost the same path.

Another type of graph that Filter-Kruskal algorithm has

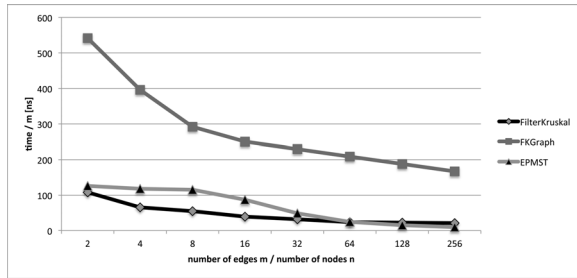


Fig. 3. Time per edge for graphs with 2^{16} nodes and random edge weights.

used is the so called *difficult instance*. The graph is also generated by producing random edge weights. An instance of this graph is such that each edge will call for the expensive decreaseKey operation in Prim's algorithm. FKGraph takes much more time than FilterKruskal and EPMST. This is because of the difference between our input data sets. FilterKruskal works better when the number of edges per node is at most 64. After

that point, EPMST takes less time. These comparisons have been shown in Fig. 3. Our EPMST consumes 14.78 ns per edge whereas FilterKruskal takes 22.49 ns when the edge per node value is 128. But this margin increases when this value is 256. FilterKruskal needs 20.96 ns for this case and EPMST takes only 9.78 ns.

Geometric graphs are generated by connecting each

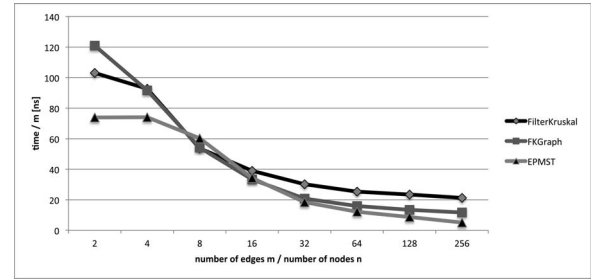


Fig. 4. Time per edge for random geometric graphs with 2^{16} nodes.

node with their k nearest neighbors. As nodes are positioned randomly and nodes are connected with their k nearest neighbors, desired edges have edge values within a range. Therefore the pivot value of our algorithm separates these edges efficiently. Fig. 4 illustrates this for random geometric graphs with 2^{16} nodes. All the three lines follow the same pattern as Fig. 1 and Fig. 2. EPMST outperforms FilterKruskal and FKGraph when the edge per node value is 16 or more.

A lollipop graph has the shape of a lollipop with a linear

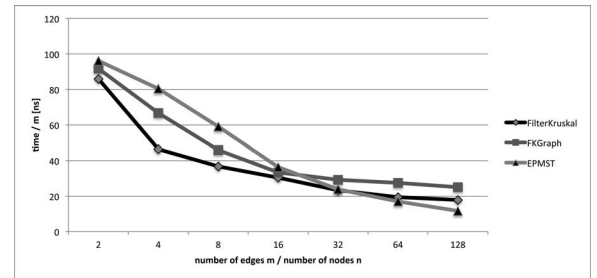


Fig. 5. Time per edge for lollipop graphs of 2^{10} nodes.

part and a circular part. This graph has a single path among $\frac{n}{2}$ nodes which is very sparse and a dense part of the remaining $\frac{n}{2}$ nodes. Therefore it is really difficult to choose a suitable pivot value to separate most of the desired edges for the Kruskal's algorithm. But we can easily identify very sparse portions of

the graph as well as the dense portions. Then we can apply the simple Kruskal algorithm on the very sparse components and EPMST on the dense components.

Fig. 5 and Fig. 6 show results for lollipop graphs, where

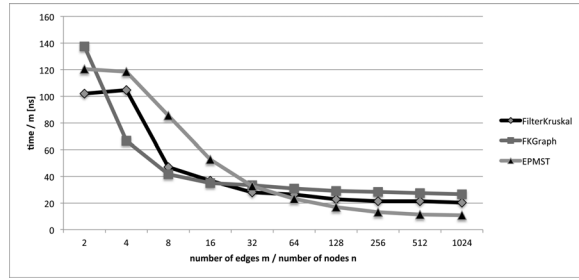


Fig. 6. Time per edge for lollipop graphs with 2^{17} nodes.

we observe a similar trend for all of the curves. For these instances EPMST outperforms the Filter-Kruskal algorithm when the numbers of edges per node is 16 or more.

All these figures illustrate that our algorithm outperforms the Filter-Kruskal algorithm very clearly for not so sparse graphs. The trend shows that if the graphs are more dense, then our algorithm works much better. When the number of edges per node increases, our algorithm outperforms Filter-Kruskal with an increasing performance gap.

EPMST performs its best for random dense graphs.

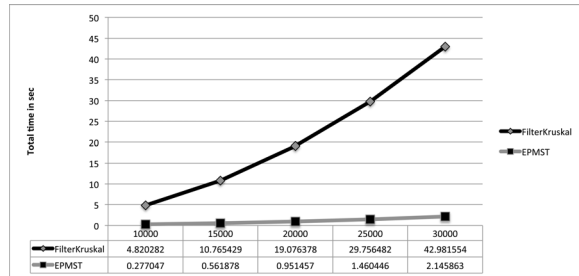


Fig. 7. Time in sec for random dense graphs with different numbers of nodes.

Fig. 7 compares the performance of the algorithm with Filter-Kruskal for random dense graphs with different numbers of nodes. X axis shows the number of nodes and Y axis shows time in seconds. For these graphs, we choose the $n \log n^{th}$ lightest edge as the pivot for EPMST. In this graph, we see that EPMST runs 20 times faster than Filter-Kruskal for complete graphs with 30,000 nodes.

We have employed five types of input graphs to compare EPMST with FilterKruskal. We provide the total time consumed by EPMST and Filter-Kruskal on dense graphs. In the other figures we show the time spent per edge to compute a minimum spanning tree. For very sparse graphs both of our algorithms do not take much time. Even any simple minimum spanning tree algorithm works fine for very sparse graphs. But our EPMST outperforms Filter-Kruskal for not too sparse graphs. The margin between these two algorithms increases with an increment on the edge per node value. Filter-Kruskal algorithm has better performances over Prim's and Kruskal's

algorithms for most of the cases [15]. Therefore we have only compared our algorithm with the Filter-Kruskal algorithm.

V. CONCLUSIONS

Edge Pruned Minimum Spanning Tree (EPMST) algorithm uses the benefits of Kruskal's and Prim's algorithms. As Kruskal's algorithm performs better on sparse graphs and produces a minimum spanning forest, EPMST filters out a large number of edges before running the Kruskal's algorithm. Then it runs Prim's algorithm on a compact dense graph. Selection of pivot value is a major concern here. [15] makes a series of tests to prove the superiority of their recently devised Filter-Kruskal algorithm. We observe that our algorithm outperforms the Filter-Kruskal algorithm on a wide variety of graph types.

REFERENCES

- [1] S. Meguerdichian, F. Koushanfar, M. Potkonjak, and M. Srivastava, "Coverage problems in wireless ad-hoc sensor networks," in *INFOCOM 2001. Twentieth Annual Joint Conference of the IEEE Computer and Communications Societies. Proceedings. IEEE*, vol. 3, 2001, pp. 1380–1387 vol.3.
- [2] Y.-C. Tseng, T. T.-Y. Juang, and M.-C. Du, "Building a multicasting tree in a high-speed network," *Concurrency, IEEE*, vol. 6, no. 4, pp. 57–67, 1998.
- [3] S.-Q. Zheng, J. S. Lim, and S. S. Iyengar, "Routing using implicit connection graphs [vlsi design]," in *VLSI Design, 1996. Proceedings., Ninth International Conference on*. IEEE, 1996, pp. 49–52.
- [4] Y. Maon, B. Schieber, and U. Vishkin, "Parallel ear decomposition search (eds) and st-numbering in graphs," in *VLSI Algorithms and Architectures*. Springer, 1986, pp. 34–45.
- [5] G. L. Miller and V. Ramachandran, "Efficient parallel ear decomposition with applications," *Manuscript, UC Berkeley, MSRI*, vol. 135, 1986.
- [6] R. E. Tarjan and U. Vishkin, "An efficient parallel biconnectivity algorithm," *SIAM Journal on Computing*, vol. 14, no. 4, pp. 862–874, 1985.
- [7] M. Brinkhuis, G. A. Meijer, P. J. van Diest, L. T. Schuurmans, and J. Baak, "Minimum spanning tree analysis in advanced ovarian carcinoma. an investigation of sampling methods, reproducibility and correlation with histologic grade." *Analytical and quantitative cytology and histology/the International Academy of Cytology [and] American Society of Cytology*, vol. 19, no. 3, pp. 194–201, 1997.
- [8] K. Kayser, S. Jacinto, G. Böhm, P. Fritz, W. Kunze, A. Nehrlisch, and H. Gabius, "Application of computer-assisted morphometry to the analysis of prenatal development of human lung," *Anatomia, histologia, embryologia*, vol. 26, no. 2, pp. 135–139, 1997.
- [9] K. Kayser, H. Stute, and M. Tacke, "Minimum spanning tree, integrated optical density and lymph node metastasis in bronchial carcinoma." *Analytical cellular pathology: the journal of the European Society for Analytical Cellular Pathology*, vol. 5, no. 4, pp. 225–234, 1993.
- [10] M. Matos, B. N. Raby, J.-M. Zahm, M. Polette, P. Birembaut, and N. Bonnet, "Cell migration and proliferation are not discriminatory factors in the in vitro sociologic behavior of bronchial epithelial cell lines," *Cell motility and the cytoskeleton*, vol. 53, no. 1, pp. 53–65, 2002.
- [11] C. Chen and S. Morris, "Visualizing evolving networks: Minimum spanning trees versus pathfinder networks," in *Information Visualization, 2003. INFOVIS 2003. IEEE Symposium on*. IEEE, 2003, pp. 67–74.
- [12] V. Jarník, "O jistém problému minimálním," *Práce Moravské Přírodovědecké Společnosti*, vol. 6, pp. 57–63, 1930.
- [13] R. C. Prim, "Shortest connection networks and some generalizations," *Bell system technical journal*, vol. 36, no. 6, pp. 1389–1401, 1957.
- [14] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proceedings of the American Mathematical society*, vol. 7, no. 1, pp. 48–50, 1956.
- [15] V. Osipov, P. Sanders, and J. Singler, "The filter-kruskal minimum spanning tree algorithm." in *ALENEX*, vol. 9. SIAM, 2009, pp. 52–61.
- [16] J. Brennan, "Minimal spanning trees and partial sorting," *Operations Research Letters*, vol. 1, no. 3, pp. 113–116, 1982.

- [17] R. Paredes and G. Navarro, "Optimal incremental sorting," in *ALENEX*. SIAM, 2006, pp. 171–182.
- [18] A. Kershenbaum and R. Van Slyke, "Computing minimum spanning trees efficiently," in *Proceedings of the ACM annual conference-Volume 1*. ACM, 1972, pp. 518–527.
- [19] J. Katajainen and O. Nevalainen, "An alternative for the implementation of kruskal's minimal spanning tree algorithm," *Science of Computer Programming*, vol. 3, no. 2, pp. 205–216, 1983.
- [20] B. Bollobás, *Random graphs*. Springer, 1998.
- [21] B. M. Moret and H. D. Shapiro, "An empirical assessment of algorithms for constructing a minimum spanning tree," *DIMACS Monographs in Discrete Mathematics and Theoretical Computer Science*, vol. 15, pp. 99–117, 1994.